MÓDULO 5

Arquitectura Limpia y Hexagonal

El arte del desacoplamiento: Diseño de sistemas mantenibles mediante el aislamiento estricto de la lógica de negocio.

NotebookLM

Qué hace el sistema

Gestión de Usuarios

Procesamiento de Pagos

Motor de Reglas

NÚCLEO DEL NEGOCIO

Inventario y Catálogo

Motor de Procemos

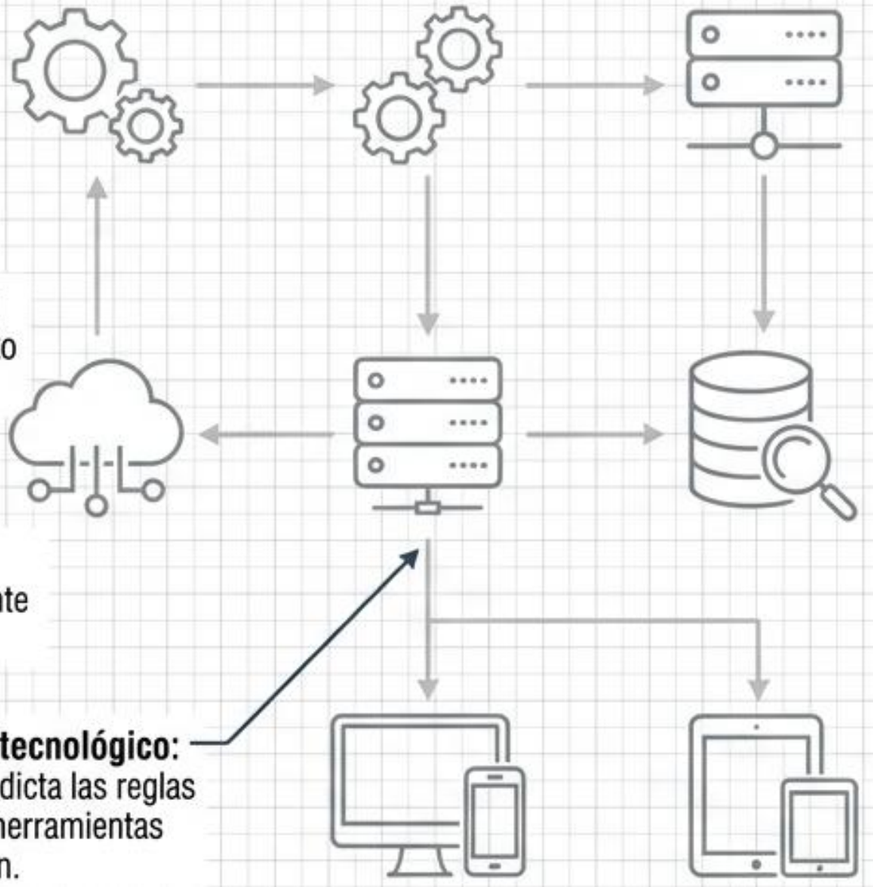
Análisis de Datos

Cómo lo hace

Separación de preocupaciones (SoC): Cada módulo asume una responsabilidad lógica única y aislada.

Aislamiento del núcleo: El valor real del negocio reside en el centro, intacto ante los cambios tecnológicos.

Jerarquía de flujo: Organización estructural para controlar estrictamente la dirección de los datos.



Agnosticismo tecnológico: La arquitectura dicta las reglas operativas; las herramientas solo las ejecutan.

Nota del Arquitecto: El objetivo no es que el sistema funcione hoy, sino que sea predecible y económico de modificar durante los próximos 10 años.

NotebookLM

Independencia del Framework: Las librerías son plugins reemplazables, no restricciones inmutables.

Independencia de la UI: Web, consola o móvil son meros canales de entrega transitorios.



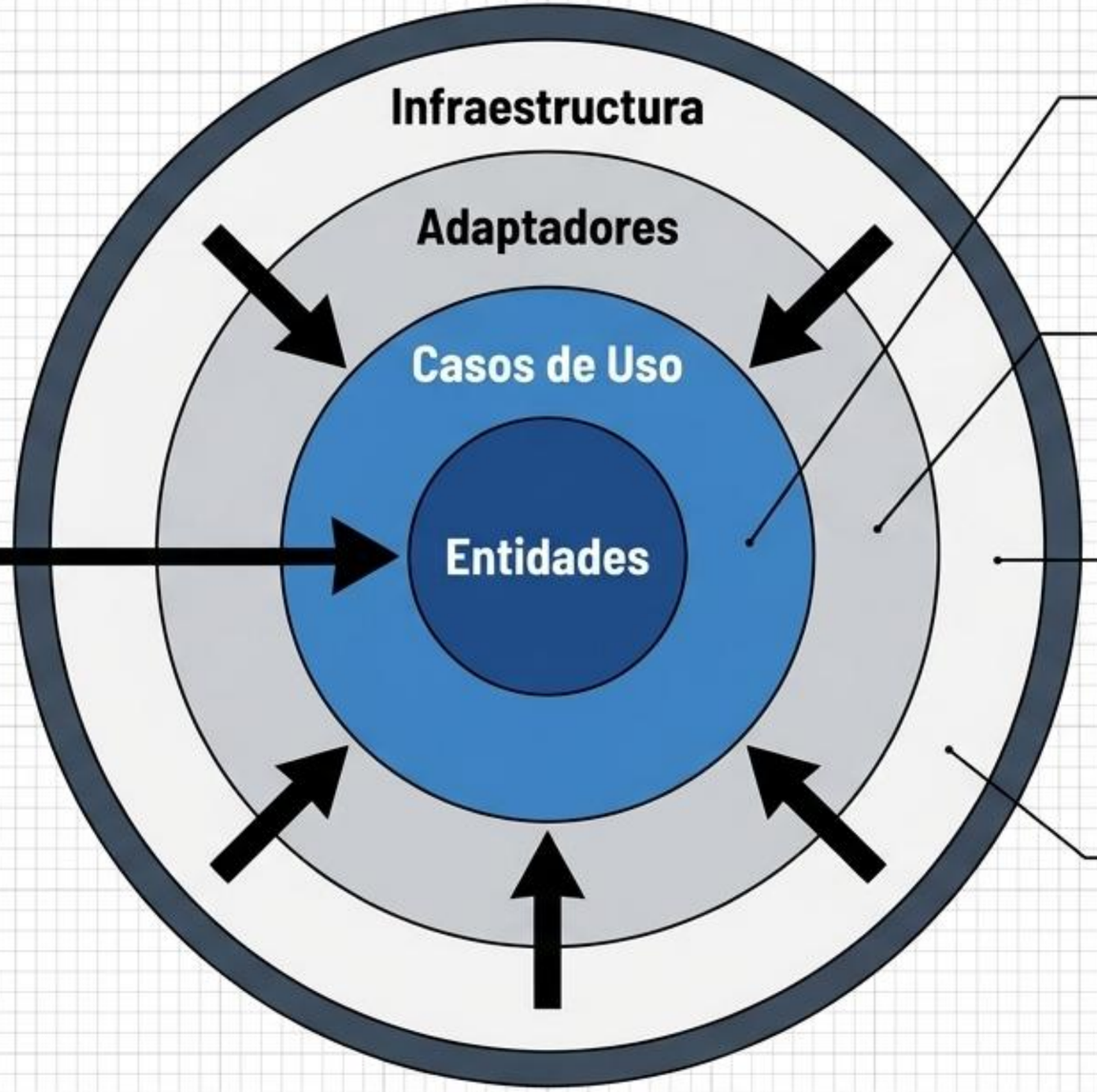




Independencia de la Base de Datos: SQL, NoSQL o memoria son detalles de persistencia intercambiables.

Postergación de Decisiones: Todo lo ajeno a la regla de negocio crítica se considera un detalle periférico.

NotebookLM



Flujo de Dependencias

Entidades: Reglas empresariales críticas, universales e independientes.

Casos de Uso: Orquestación de la lógica de aplicación específica del sistema.

Adaptadores: Traductores entre formatos externos e internos.

Regla de Dependencia: Nada en un círculo interno puede referenciar un elemento externo.

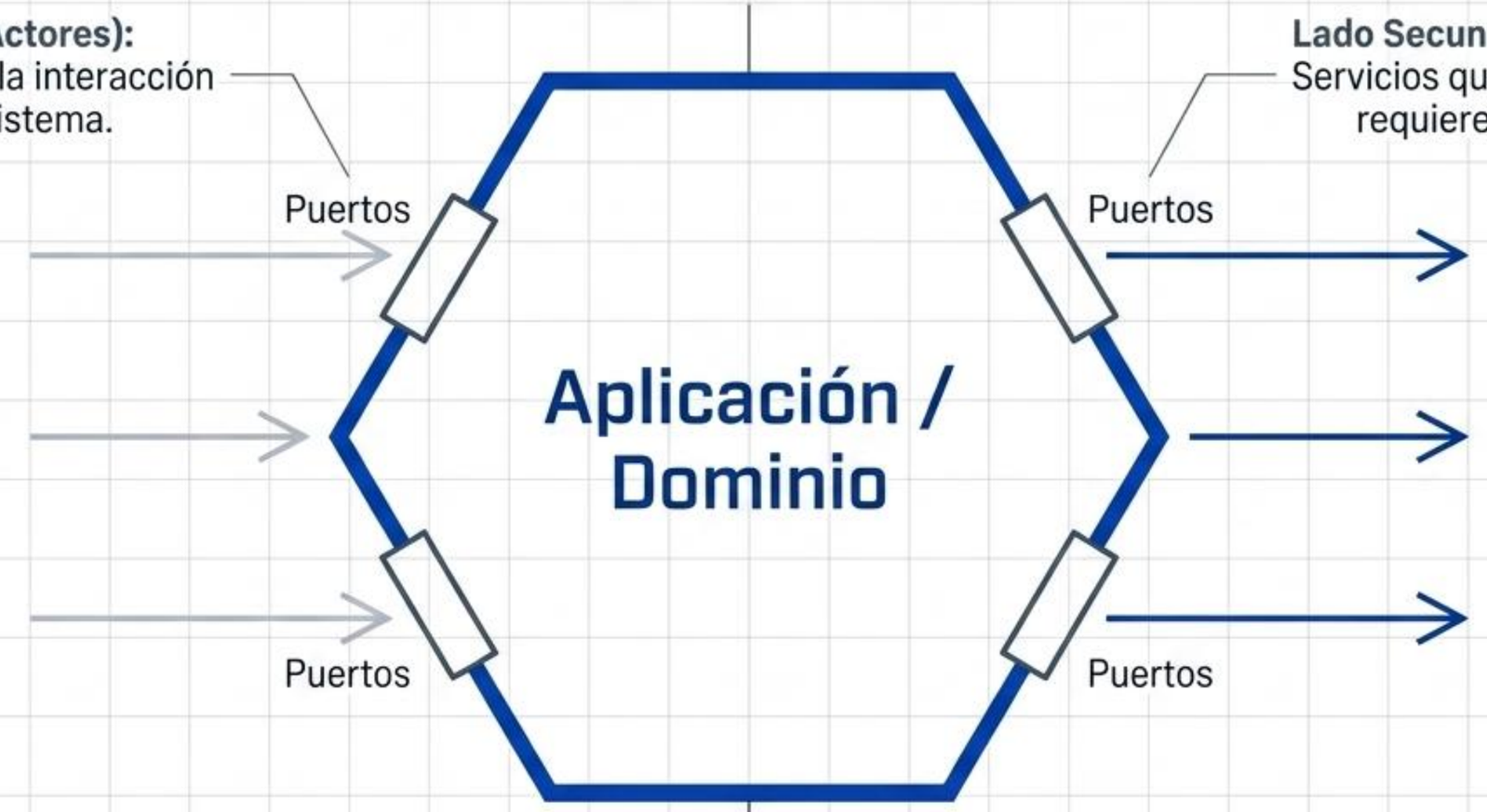
Rigor Técnico: El código fuente interno no debe mencionar dependencias externas. Este flujo unidireccional garantiza el ciclo de vida a largo plazo del software.

NotebookLM

Frontera Hermética:
El hexágono encapsula y protege estrictamente el núcleo de negocio.

Lado Primario (Actores):
Entidades que inician la interacción y conducen el sistema.

Lado Secundario (Dependencias):
Servicios que el sistema conduce y requiere para completarse.

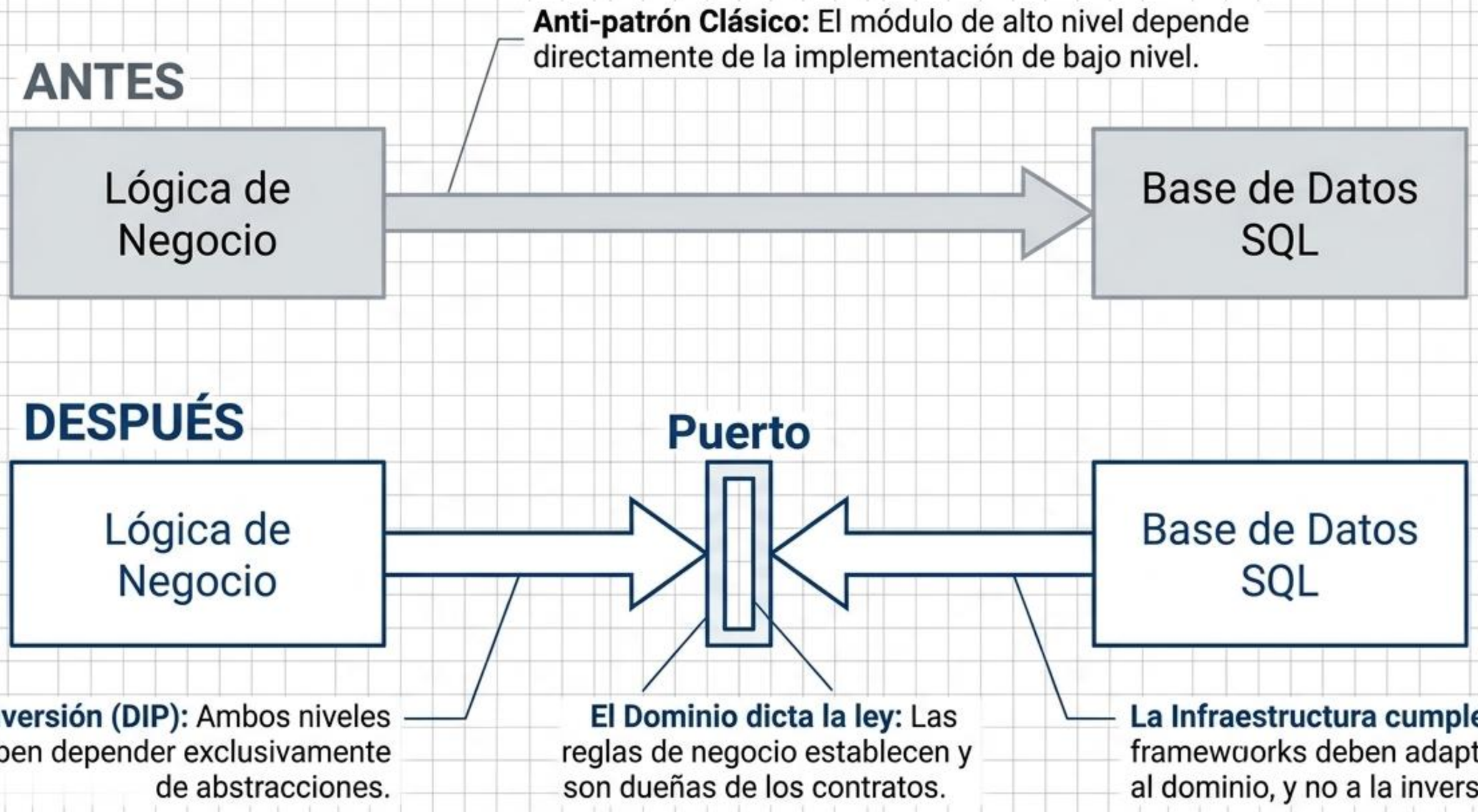


Simetría Arquitectónica:
Se aplica la misma filosofía de aislamiento para entradas y salidas.

NotebookLM

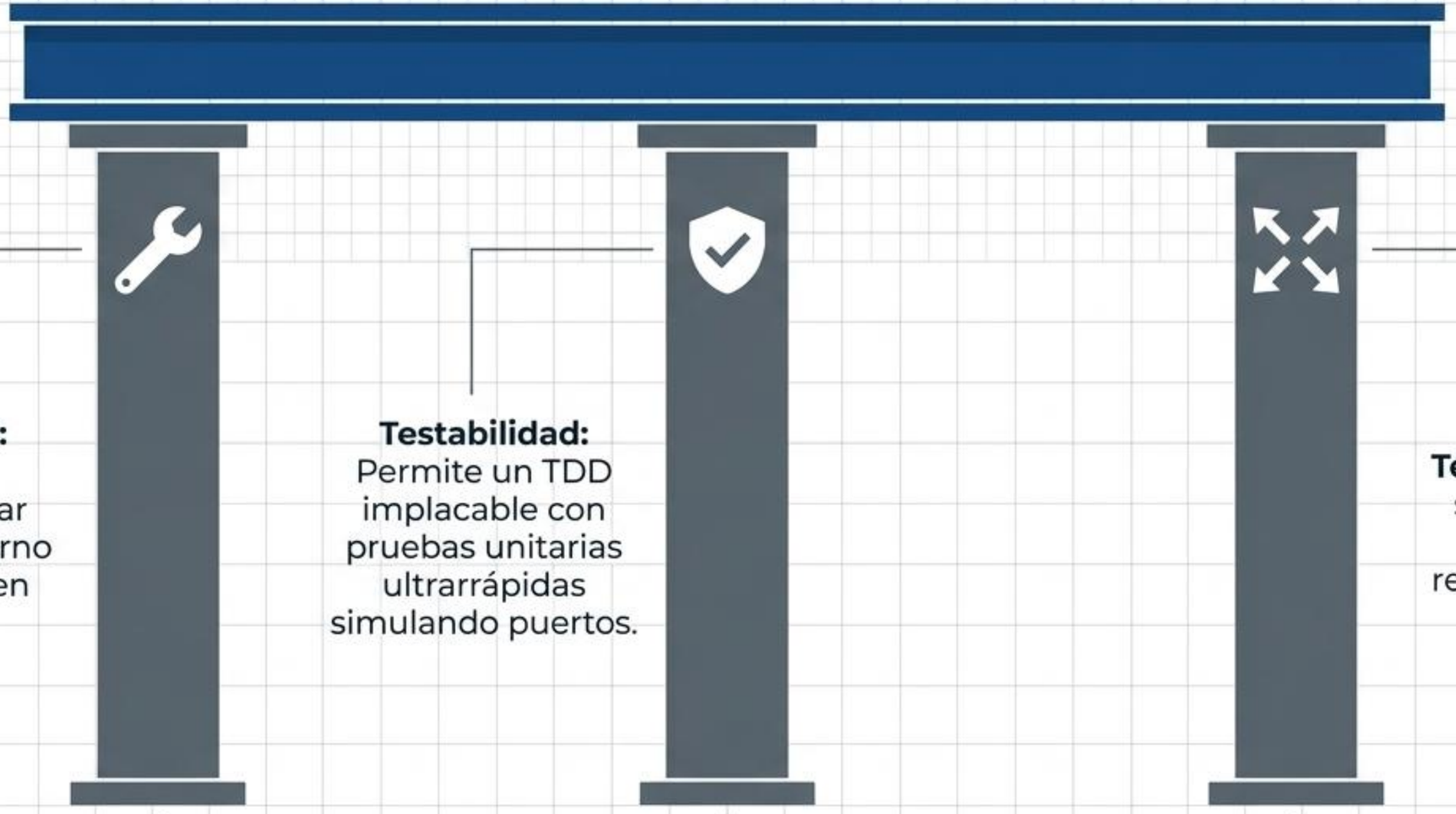
	 **PUERTO** (La Abstracción)	 **ADAPTADOR** (La Implementación)
Naturaleza	Interfaz abstracta o Contrato	Clase concreta y código técnico
Ubicación	Dentro del hexágono (Dominio)	Fuera del hexágono (Infraestructura)
Propósito	Define QUÉ necesita o expone el sistema	Define CÓMO se realiza la comunicación técnica
Regla de Diseño	Propiedad de la lógica de negocio	Proxy periférico totalmente intercambiable

NotebookLM



Nota del Arquitecto: Al invertir la dependencia mediante interfaces polimórficas (Puertos), obligamos a que el detalle técnico (SQL) dependa de la abstracción. La base de datos es ahora un mero plugin pasivo de nuestro dominio.

NotebookLM



Mantenibilidad: Cambios localizados. Alterar un adaptador externo no genera fallos en cascada en el núcleo.

Testabilidad: Permite un TDD implacable con pruebas unitarias ultrarrápidas simulando puertos.

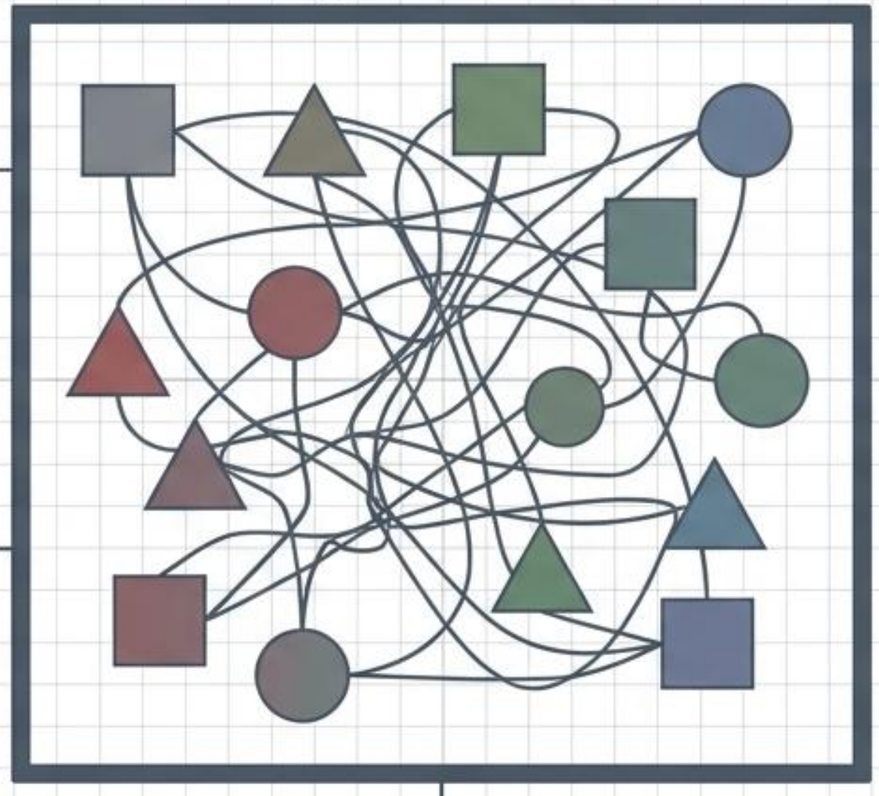
Flexibilidad Técnica: Adopción segura y ágil de nuevos requerimientos no funcionales.

Escalado Organizacional: Múltiples equipos pueden trabajar en paralelo sobre distintos adaptadores sin colisiones estructurales.

NotebookLM

Definición: Medida de cuán intrínsecamente relacionadas están las responsabilidades dentro de un mismo módulo.

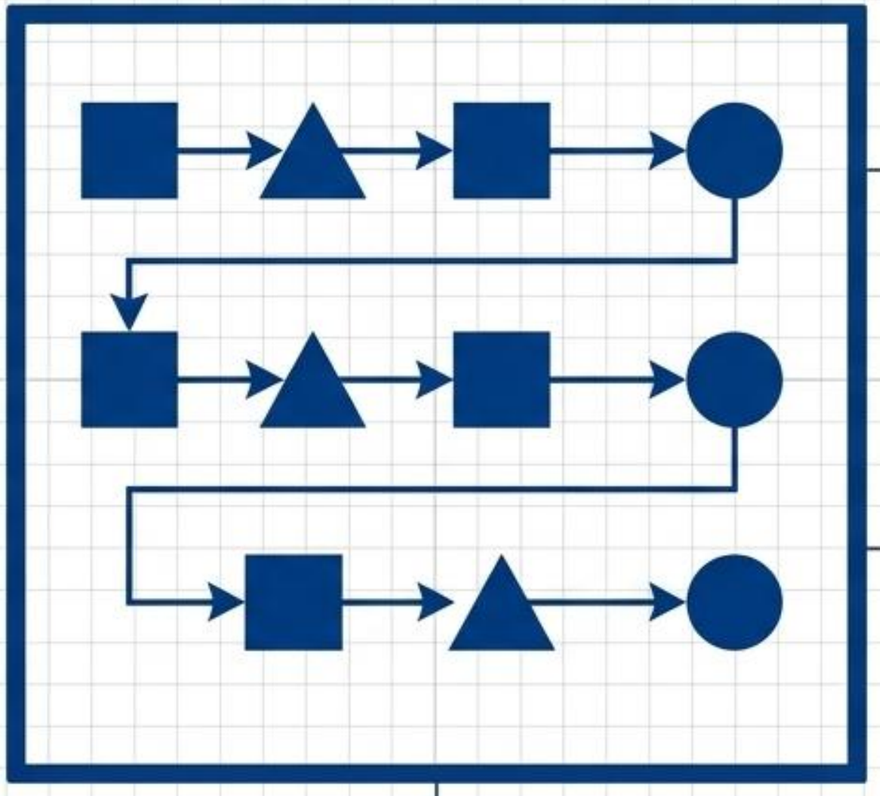
Baja Cohesión



Cohesión Funcional: Todos los elementos internos contribuyen a la ejecución de una única tarea crítica.

El Objetivo: Agrupar elementos que cambian por la misma razón y exactamente al mismo tiempo.

Alta Cohesión Funcional

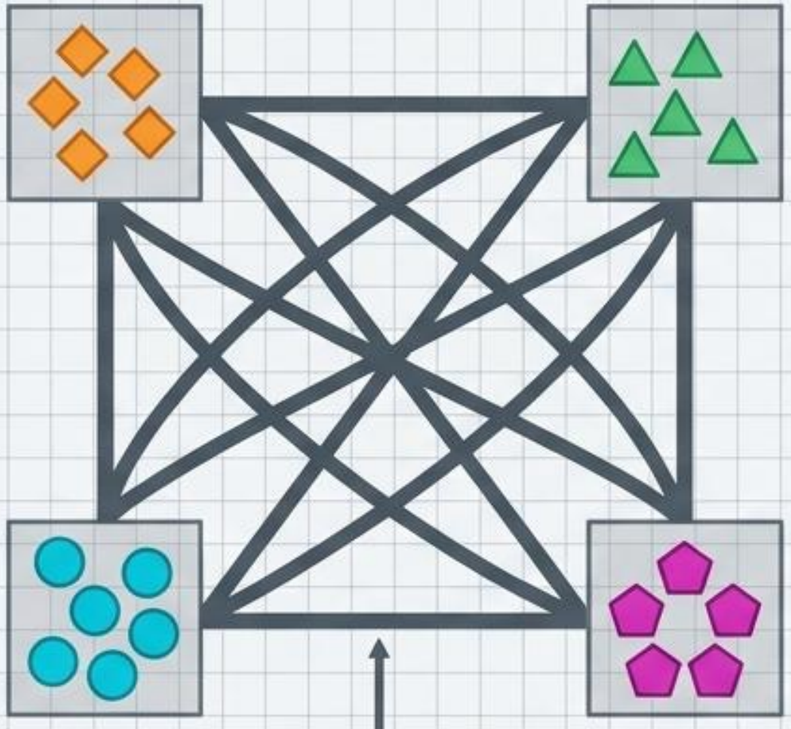


Principio SOLID: Es el Single Responsibility Principle elevado a una escala arquitectónica.

NotebookLM

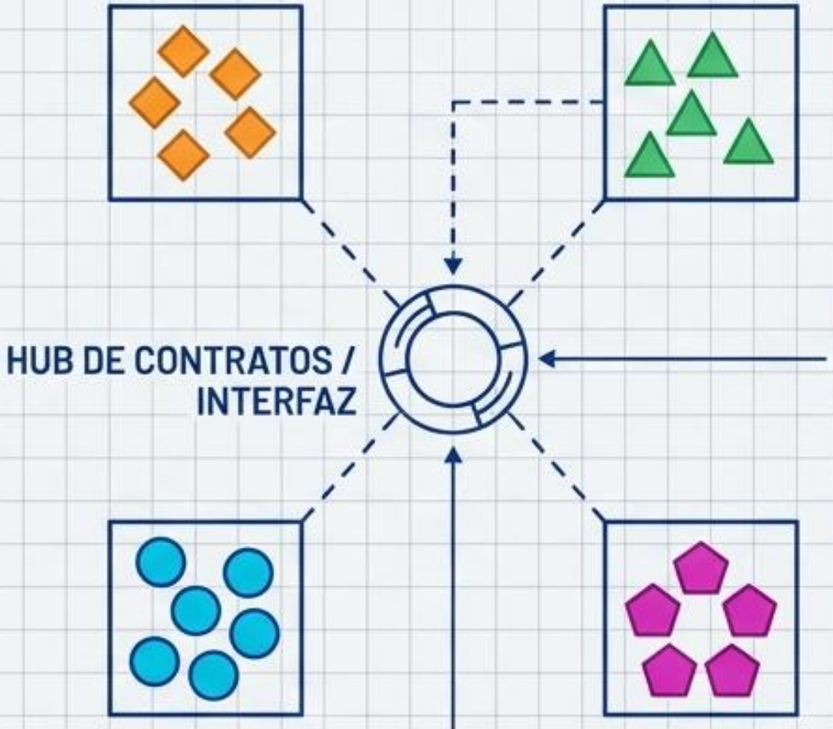
Arquitectura: Contrastes de Acoplamiento

Alto Acoplamiento



Riesgo Estructural: El alto acoplamiento crea sistemas rígidos donde un cambio mínimo causa fallos en cascada.

Bajo Acoplamiento



Abstracción: Los componentes se comunican exclusivamente mediante contratos (interfaces), jamás mediante implementaciones.

Inyección (DI): Las dependencias operativas se proveen desde el exterior en tiempo de ejecución.

Mensajería: Desacoplamiento temporal y espacial utilizando colas asíncronas o eventos.

NotebookLM



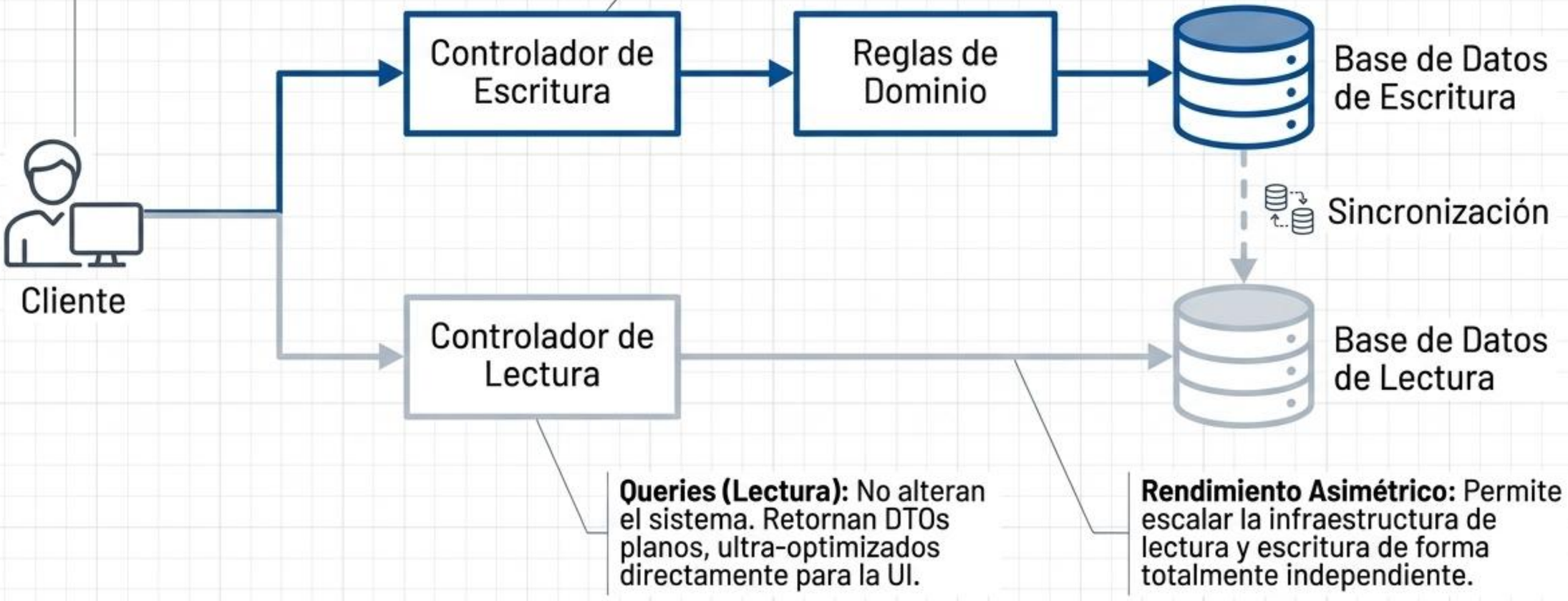
Nota del Arquitecto: Esta matriz es la brújula absoluta. Cada decisión técnica y refactorización debe tener el único propósito de mover la base de código hacia la esquina superior izquierda.

NotebookLM

Arquitectura: Segregación de Responsabilidad de Comando y Consulta (CQRS)

CQRS: Segregación estricta de la responsabilidad entre Comandos y Consultas.

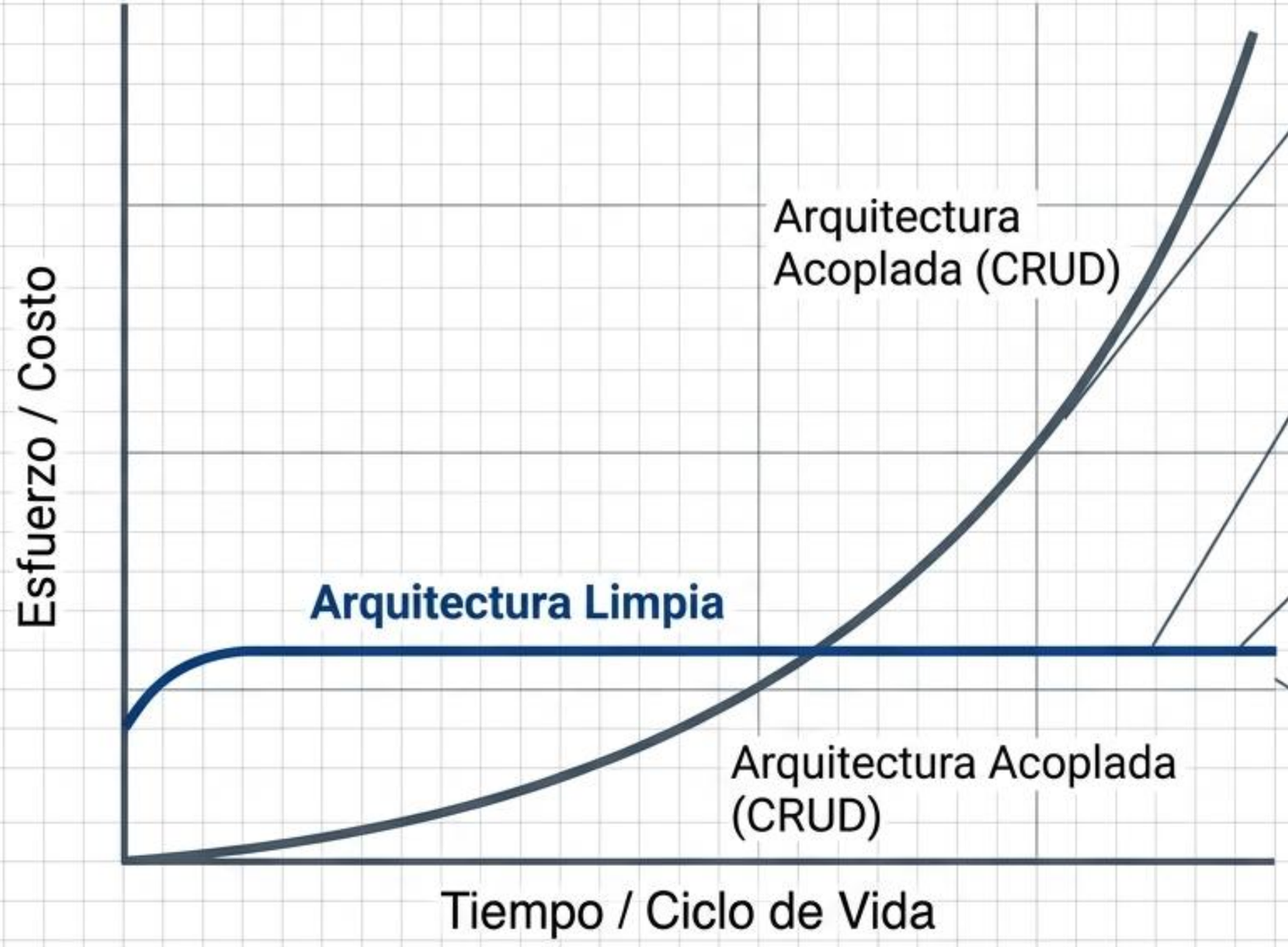
Commands (Escritura): Mutan el estado. Pasan por una validación implacable del Dominio y reglas de negocio.



Queries (Lectura): No alteran el sistema. Retornan DTOs planos, ultra-optimizados directamente para la UI.

Rendimiento Asimétrico: Permite escalar la infraestructura de lectura y escritura de forma totalmente independiente.

NotebookLM



El Impuesto Inicial:
Exige mayor rigor analítico, abstracciones formales y código base al comenzar.

La Curva de Retorno:
Previene la parálisis técnica; agregar la funcionalidad #100 cuesta lo mismo que la #1.

Resiliencia al Cambio:
Los frameworks y bases de datos caducan; las reglas de negocio aisladas perduran.

El Veredicto:
Inversión estructural obligatoria para proyectos de larga duración, alta complejidad y escalado de equipos.

NotebookLM